

FUNDAMENTOS DE TEORÍA DE LA COMPUTACIÓN



Docentes: Esp. Lic. Nicolás Perez

Contactos: nperez_dcao_smn@outlook.com

Slack

https://join.slack.com/t/fundamentosde-hej6488/shared_invite/zt-2ofgyh30i-TXnEme1um9ddYNa3JBw1tg

Lista de reproducción

<https://www.youtube.com/playlist?list=PLbwWly6AYXqd4YxGe1nzzgD89-I5MfNIUQ>

Material digital

https://drive.google.com/drive/folders/115GpnFMmMX02DvuLkl8lsMXVuV_soMHD?usp=drive_link

Meet

<https://meet.jit.si/NykolaiPerezUNLa>

Tabla de Contenidos

Alfabetos y Lenguajes	3
Conceptos Básicos.....	3
Lenguaje Universal.....	4
Operaciones Sobre Cadenas.....	5
Operaciones y Relaciones Entre Lenguajes	7
Igualdad Entre Lenguajes	12
Ejercicios de Ejemplo Resueltos.....	17
Gramáticas Formales	23
Definición Formal de Gramática.....	23
Reglas de Producción.....	25
Derivación de Cadenas	25
Fórmula Generada.....	26
Jerarquía de las Gramáticas.....	26
Gramáticas de Tipo 0.....	27
Gramáticas de Tipo 1.....	28
Gramáticas de Tipo 2.....	30
Gramáticas de Tipo 3.....	32
Gramáticas equivalentes.....	33
Gramáticas en Lenguajes de Programación	34
Teoría de autómatas.....	38
Componentes	39
Tabla de transiciones.....	40
Componentes	40
Diagrama de estados.....	41
Tipos de autómatas	42
Reglas Chomsky	43
Resolución ejercicio.....	47
Resolución ejercicio de aceptación de expresión regular.....	49
Eficiencia de Algoritmos.....	51
Complejidad Espacial	51
Complejidad Temporal.....	51
Análisis Empírico	51
Análisis Teórico	54
Notación Big O	56
Resolución Ejercicios.....	61
Referencias.....	67

Alfabetos y Lenguajes

En esta primera sección, se abordan conceptos básicos sobre alfabetos y lenguajes, esenciales para el estudio de la teoría de lenguajes formales y autómatas. Estos conceptos, basados en las contribuciones de Jurado Málaga (2008) y Dean Kelley (1995), son clave para entender y analizar estructuras lingüísticas y computacionales.

Este análisis no solo aclarará conceptos teóricos, sino que también demostrará su importancia en aplicaciones computacionales, mostrando cómo las estructuras formales de los lenguajes influyen en la resolución de problemas tecnológicos complejos.

Conceptos Básicos

De acuerdo a Jurado Málaga, Elena (2008), podemos definir los siguientes conceptos:

Alfabeto: conjunto no vacío y finito de símbolos. A estos símbolos también se les suele llamar letras del alfabeto. Se denota con la letra griega Σ . Como por ejemplo: $\Sigma_1 = \{a, b, c, \dots, z\}$ $\Sigma_2 = \{0, 1\}$.

Palabra: secuencia finita de símbolos de un alfabeto. Lo correcto es hablar de “palabras definidas sobre un alfabeto”. Habitualmente utilizamos en nuestros ejemplos las últimas letras minúsculas de nuestro alfabeto (x, y, z) para denotar a las palabras. Como, por ejemplo:

$x = \text{casa}$ es una palabra definida sobre el alfabeto Σ_1 .

$y = 010100$ es una palabra definida sobre el alfabeto Σ_2 .

Cadena: Toda comunicación conlleva la utilización de un lenguaje, que podemos definir como un conjunto de palabras (también llamadas cadenas) formadas por símbolos de un alfabeto.

Lenguaje: Un lenguaje L definido sobre un alfabeto Σ , es un conjunto cualquiera de palabras definidas sobre dicho alfabeto, por lo tanto, $L \subset \omega(\Sigma)$.

Además, de acuerdo a Dean Kelley (1995) definimos los siguientes conceptos:

Elemento: puede definirse como una secuencia de símbolos de longitud finita, sin restricciones en cuanto a dicha longitud.

Cadena vacía: la cual se denota por el símbolo λ , es una palabra sobre cualquier alfabeto. La palabra vacía es una secuencia vacía de símbolos tomados de cualquiera que sea el alfabeto en cuestión.

Lenguaje vacío: Dado que un lenguaje es un conjunto de cadenas, se puede tener el lenguaje compuesto por ninguna cadena — el lenguaje vacío. Este no es el mismo lenguaje que el que consta de la cadena vacía $\{\lambda\}$. El lenguaje vacío se denota de la misma forma que el conjunto vacío, ϕ .

Lenguaje Universal

De acuerdo a Jurado Málaga, Elena (2008) un Lenguaje Universal definido sobre un alfabeto es el conjunto de todas las palabras que se pueden construir con las letras de dicho alfabeto. Se denota por $\omega(\Sigma)$. El lenguaje universal de cualquier alfabeto es infinito, y siempre pertenece a él la palabra vacía.

Ejemplo: si $\Sigma = \{a\}$ entonces $\omega(\Sigma) = \{\lambda, a, aa, aaa, \dots\}$

En cambio, Kelley, representa el lenguaje universal del alfabeto Σ con el símbolo Σ^* (1995, p. 45).

Operaciones Sobre Cadenas

Longitud de una cadena: Según Jurado Málaga, Elena (2008) el término "cadena" y "palabra" pueden ser considerados equivalentes. A partir de esta equivalencia, el autor define Longitud de una palabra: es el número de símbolos que componen la palabra. Se representa utilizando dos barras verticales ($||$). Ejemplos:

$$x = abcd \quad |x| = 4 \quad y = banana \quad |y| = 6$$

Propiedades:

$$|wz| = |w| + |z|$$

$$|\lambda| = 0$$

Concatenación: Kelley (1995) indicó que:

Si w y z son cadenas, la concatenación de w con z es la cadena que se obtiene al añadir a la cadena w la palabra z . Por ejemplo, si $w = \text{"banana"}$ y $Z = \text{"rama"}$, la concatenación de w con z es la cadena "bananarama". La concatenación de las palabras w y z se denota como wz o $w.z$. (p. 32)

Potencia de una palabra sobre un alfabeto: Kelley (1995) indicó que:

Sea w una palabra; para $n \in \mathbb{N}$ se define

$$w^n = \begin{cases} \varepsilon, & \text{si } n = 0 \\ ww^{n-1}, & \text{si } n > 0 \end{cases}$$

Por tanto, si $w = 122$ sobre el alfabeto $L = \{1,2\}$, se tiene

$$w^0 = \varepsilon$$

$$w^1 = 122$$

$$w^2 = 122122$$

$$w^3 = 122122122$$

y así sucesivamente. Se dice que w^i es la potencia i -ésima de w (p. 32)

Potencia i -ésima de una palabra (x^i) Consiste en concatenar una palabra x consigo misma, i veces. $x^i = x \dots_i x$ (Jurado, 2008, p.17)

Igualdad entre cadenas: Si w y z son palabras, se dice que w es igual a z , si tienen la misma longitud y los mismos símbolos en la misma posición. Se denota mediante $w=z$ (Kelley, 1995, p. 32).

Prefijo, sufijo y prefijo propio: De acuerdo a Dean Kelley (1995):

Las nociones de sufijo y prefijo de cadenas sobre un alfabeto son análogas a las que se usan habitualmente. Si w y z son palabras, se dice que x es prefijo de w , si para alguna cadena y se obtiene que $w = xy$. Por ejemplo, si w es la cadena 121, entonces la cadena $x = 12$ es un prefijo de w , e $y = 1$. Si se considera $y = \epsilon$, entonces para $w = xy$ se tiene que $w = x$, con lo que toda palabra puede considerarse prefijo de sí misma. (p. 32)

El sufijo es lo opuesto del prefijo, es una subcadena que representa la segunda parte de la cadena original. Siguiendo el ejemplo del prefijo, si tenemos la cadena $w = 121$ y la cadena $x = 12$ que sería el prefijo, entonces la cadena $y = 1$ sería el sufijo, aunque también podría ser el sufijo una cadena $y = 21$ junto con el prefijo $x = 1$

Dean Kelley (1995) define el prefijo propio en la siguiente cita:

Introduciremos el término prefijo propio para denotar aquellas cadenas que son prefijos de una palabra, pero no iguales a la misma. Por eso $x = 121$ es un prefijo de la cadena $w = 121$, pero no es un prefijo propio de w . (pp. 32-33)

Subcadena o subpalabra: De acuerdo a Dean Kelley (1995):

Una cadena w es una subcadena o subpalabra de otra cadena z si existen las cadenas x e y para las cuales $z = xwy$. (p. 33)

Inversa o traspuesta de una palabra:

De acuerdo a Dean Kelley (1995):

La inversa o transpuesta de una palabra w es la imagen reflejada de w . Por ejemplo, si $w = \text{"able"}$ entonces su inversa es "elba" . Para denotar la inversa de w se usa w^I . Una definición más precisa de la misma puede ser la siguiente:

$$w^I = \{w, \quad \text{si } w = \varepsilon \text{ y } a, \quad \text{si } w = ay \text{ por lo tanto } a \in \Sigma \text{ y } y \in \Sigma^* \}$$

Jurado nos da una definición similar, con la diferencia de que representa a la inversa de una cadena con el símbolo x^{-1} en vez de x^I como lo hace Kelley, además de que nos menciona "Reflexión" como otro posible nombre para la inversa de una palabra (2008, p. 17)

Operaciones y Relaciones Entre Lenguajes

Concatenación de Lenguajes: Kelley nos define al lenguaje concatenación en la siguiente cita:

Sean A y B lenguajes sobre un alfabeto. Se define el lenguaje concatenación de A y B como

$$A \cdot B = \{w \cdot x \mid w \in A \text{ y } x \in B\}$$

Por tanto, $A \cdot B$ está formado por todas las cadenas que se forman concatenando cada cadena de A con todas las cadenas de B . (1995, p. 34)

Nos reafirma esta definición Jurado con la suya:

Sean dos lenguajes L_1 y L_2 definidos sobre el mismo alfabeto Σ , la concatenación de ambos lenguajes estará formada por todas las palabras obtenidas al concatenar una palabra cualquiera de L_1 con otra de L_2 . (2008, p. 18)

Aunque ambos autores difieren en una cosa. Jurado sostiene en la cita anterior que la concatenación se realiza entre dos lenguajes que comparten un mismo alfabeto, mientras que Kelley sostiene que no es necesario que este sea el caso, sino que el lenguaje resultado de la operación se encuentra definido sobre la unión de los alfabetos originales (1995, p. 34)

Un ejemplo de concatenación sería el siguiente, si tenemos los lenguajes

$$L_1 = \{sol, mar\} \quad L_2 = \{eado, ía\}$$

La concatenación de estos lenguajes nos daría el siguiente resultado

$$L_1 \cdot L_2 = \{soleado, solía, mareado, maría\}$$

Jurado define las siguientes propiedades de la **concatenación** de lenguajes (2008, p. 18):

- **Asociativa**

En caso de haber múltiples concatenaciones seguidas, no importa el orden en el que se realizan no se afecta el resultado $L_1 \cdot (L_2 \cdot L_3) = (L_1 \cdot L_2) \cdot L_3$

- **No es Conmutativa**

El orden de los lenguajes afecta al resultado de la unión $L_2 \cdot L_1 \neq L_1 \cdot L_2$

- **Su elemento neutro es el Lenguaje con carácter vacío ($L = \{\lambda\}$)**

La unión con un lenguaje que solo contenga el carácter vacío no modifica al lenguaje original $L \cdot L_\lambda = L$

Potencia de Lenguajes:

Al igual que con las cadenas, la potencia de un lenguaje consiste en la concatenación de este con sí mismo n cantidad de veces. Kelley lo define con la siguiente fórmula:

$$A^n = \{\{\epsilon\}, \quad \text{si } n = 0 \quad A \cdot A^{n-1}, \quad \text{si } n \geq 1$$

Por ejemplo, si tenemos este lenguaje:

$$L = \{sol, mar\}$$

Y lo potenciamos por dos el resultado sería el siguiente:

$$L^2 = \{sol\,sol, sol\,mar, mar\,sol, mar\,mar\}$$

Estas son algunas propiedades de la potencia de lenguajes que nos presenta Jurado (2008, p. 18):

- **Suma de potencias**

$$L^i \cdot L^j = L^{i+j}$$

- **Potencia 0**

Un lenguaje potenciado por 0 equivale al lenguaje con palabra vacía $L^0 = L_\lambda = \{\lambda\}$

Unión de Lenguajes:

La unión entre lenguajes es una operación que se puede ver explicada de las siguientes maneras:

Si A y B son lenguajes sobre el alfabeto I, entonces la unión de A y B se denota mediante $A \cup B$ y está formada por todas las palabras que pertenecen al menos a uno de los dos lenguajes. (Kelley, 1995, p. 35)

La unión de dos lenguajes L1 y L2 definidos sobre el mismo alfabeto Σ es otro lenguaje, también definido sobre ese alfabeto, que contiene todas las palabras de L1 y todas las de L2. (Jurado, 2008, p. 16)

La definición matemática es la siguiente:

$$L_1 \cup L_2 = \left\{ \frac{x}{x} \in L_1 \vee x \in L_2 \right\}$$

Por ejemplo, si tenemos los lenguajes

$$L_1 = \{a, b, c\} \quad L_2 = \{x, y, z\}$$

Entonces la unión sería la siguiente

$$L_1 \cup L_2 = \{a, b, c, x, y, z\}$$

Jurado define las siguientes propiedades de la unión de lenguajes (2008, p. 17):

- **Asociativa**

En caso de haber múltiples uniones seguidas, no importa el orden en el que se realizan no se afecta el resultado $L_1 \cup (L_2 \cup L_3) = (L_1 \cup L_2) \cup L_3$

- **Conmutativa**

El orden de los lenguajes no afecta al resultado de la unión $L_2 \cup L_1 = L_1 \cup L_2$

- **Su Elemento Neutro es el Lenguaje Vacío (ϕ)**

La unión con un lenguaje vacío no modifica al lenguaje original $L \cup \phi = L$

- **Idempotencia**

La unión de un lenguaje consigo mismo n veces siempre va a dar como resultado el mismo lenguaje $L \cup L = L$

Mientras que Kelley nos demuestra que la concatenación de lenguajes es distributable entre los lenguajes de una unión (1995, p. 36) y nos da la siguiente representación:

$$L_1 \cdot (L_2 \cup L_3) = L_1 \cdot L_2 \cup L_1 \cdot L_3$$

Intersección de Lenguajes:

Kelley nos define la intersección entre lenguajes con la siguiente definición: “ $A \cap B$ está formado sólo por las palabras que pertenecen a los lenguajes A y B a la vez” (1995, p. 35). También nos da una representación:

$$A \cap B = \{x \in A \wedge x \in B\}$$

Como un ejemplo, si tenemos los siguientes lenguajes:

$$L_1 = \{\lambda, sol, luna, cielo, estrellas\} \quad L_2 = \{\lambda, sol, cielo, tierra, mar\}$$

La intersección entre ambas sería la siguiente:

$$L_1 \cap L_2 = \{\lambda, sol, cielo\}$$

Si dos lenguajes no comparten ninguna cadena, su intersección da como resultado el lenguaje vacío (ϕ).

Igualdad Entre Lenguajes

De acuerdo a Dean Kelley (1995), se dice que dos lenguajes A y B son iguales si contienen exactamente las mismas cadenas, es decir, son conjuntos iguales. Se denota con $A = B$.

La relación entre los sublenguajes y la igualdad:

De acuerdo a Dean Kelley (1995), los teoremas siguientes muestran la relación que existe entre sublenguajes e igualdad:

Teorema 1: Sean A y B dos lenguajes sobre el alfabeto Σ . Entonces $A = B$ si y sólo si $A \subseteq B$ y $B \subseteq A$

Demostración:

Supongamos en primer lugar que $A = B$. Tenemos que probar que $A \subseteq B$ y $B \subseteq A$. Supongamos que $x \in A$. Puesto que A y B tienen exactamente las mismas cadenas, se obtiene que $x \in B$, de lo que se deduce que $A \subseteq B$. Análogamente, si x es una cadena que pertenece a B , entonces A y B tienen exactamente las mismas cadenas, se obtiene que $x \in A$ y por lo tanto $B \subseteq A$.

Supongamos ahora que $A \subseteq B$ y $B \subseteq A$. Esto significa que toda cadena de A está también en B y viceversa. Por tanto, A y B tienen exactamente las mismas cadenas, con lo que son iguales. (p. 32)

En resumen, se demostró que dos lenguajes, A y B , son iguales si y sólo si tienen las mismas cadenas. Esto significa que la igualdad entre lenguajes se determina por la igualdad de sus cadenas. Si dos lenguajes tienen las mismas cadenas, entonces son iguales, y si son iguales, entonces tienen las mismas cadenas.

Teorema 2: Dados los lenguajes A , B y C sobre un alfabeto Σ , se cumple que:

$$A \cdot (B \cup C) = A \cdot B \cup A \cdot C$$

$$(B \cup C) \cdot A = B \cdot A \cup C \cdot A$$

Demostración:

Probaremos primero que $A \cdot (B \cup C) \subseteq A \cdot B \cup A \cdot C$. Sea $x \in A \cdot (B \cup C)$. Entonces $x = w \cdot y$ para las cadenas $w \in A$ e $y \in B \cup C$. Puesto que $y \in B \cup C$ entonces $y \in B$ o $y \in C$. Si $y \in B$, entonces $w \cdot y \in A \cdot B$, y por lo tanto, $w \cdot y \in A \cdot B \cup A \cdot C$. Por otro lado, si $y \in C$, entonces $w \cdot y \in A \cdot C$, con lo que de nuevo tenemos que $w \cdot y \in A \cdot B \cup A \cdot C$. En ambos casos se obtiene que $A \cdot (B \cup C) \subseteq A \cdot B \cup A \cdot C$.

Para probar que $A \cdot B \cup A \cdot C \subseteq A \cdot (B \cup C)$, Supongamos que $x \in A \cdot B \cup A \cdot C$. Entonces $x \in A \cdot B$ o $x \in A \cdot C$. Si $x \in A \cdot B$, entonces $x = u \cdot v$ para las cadenas $u \in A$ y $v \in B$. Puesto que $v \in B$ entonces $v \in B \cup C$ y, por tanto, $uv \in A$ e $y \in C$. En este caso y puesto que

$y \in C$, se tiene que $y \in B \cup C$, y por tanto, $w \cdot y \in A \cdot (B \cup C)$. Luego $A \cdot B \cup A \cdot C \subseteq A \cdot (B \cup C)$. Luego por el teorema 1 se obtiene que $A \cdot (B \cup C) = A \cdot B \cup A \cdot C$. (p. 32)

Así queda demostrado que la concatenación de un lenguaje con la unión de otros dos lenguajes es igual a la unión de las concatenaciones de cada lenguaje por separado. Esto permite manipular expresiones que involucran la concatenación y unión de lenguajes de manera más flexible.

Teorema 3

En el siguiente teorema se muestra una igualdad que relaciona las cerraduras A^+ y A^*

$$A^+ = A \cdot A^* = A^* \cdot A$$

Teorema 4

El inverso de la concatenación no sólo invierte las palabras concatenadas de los lenguajes, sino que también cambia el orden de la concatenación de los lenguajes, como se muestra en el siguiente teorema.

$$(A \cdot B)^1 = B^1 \cdot A^1$$

Relación entre concatenación, intersección y unión de lenguajes:

De acuerdo a Dean Kelley (1995)

La relación que tiene la concatenación con la intersección no es tan buena como con la unión.

Generalmente, la concatenación no es distributiva con respecto a la intersección. Para verlo,

supongamos que $A = \{a, e\}$, $B = \{e\}$ y $C = \{a\}$. Obsérvese que $A \cdot B = \{a, e\}$ y $A \cdot C = \{a^2, a\}$, por lo tanto $A \cdot B$

$\cap A \cdot C = \{a\}$. Por otro lado, $B \cap C = \emptyset$, con lo que $A \cdot (B \cap C) = \emptyset$.

Cierre de Kleene y cierre positivo:

De acuerdo a Jurado Málaga, Elena (2008)

Clausura (o cierre de Kleene) La clausura de un lenguaje (L^*) es el resultado de unir todas las potencias de dicho lenguaje, es decir,

$$L^* = \cup_{i=0}^{\infty} L^i$$

Clausura positiva La clausura positiva de un lenguaje (L^+) es la unión de todas las potencias de ese lenguaje, exceptuando la potencia cero.

$$L^+ = \cup_{i=1}^{\infty} L^i$$

Propiedades:

$$L^* = L^+ \cup \{\lambda\}$$

$$L^+ = L \cdot L^* = L^* \cdot L$$

$$w(\Sigma) = \Sigma^*$$

En este caso el alfabeto es considerado como un lenguaje, concretamente el formado por todas las cadenas de longitud 1.

$$\Sigma^+ = \Sigma^* \setminus \{\lambda\}$$

Diferencia entre lenguajes:

De acuerdo a Dean Kelley (1995)

La diferencia entre lenguajes corresponde a todos los elementos del primer lenguaje que no estén en el segundo. Ya que se utiliza el mismo concepto que para diferencia entre conjuntos, se puede decir que para un lenguaje A y un lenguaje B, la diferencia A - B sería:

$$A - B = \{x \mid x \in A \text{ y } x \notin B\}$$

Complemento de un lenguaje: De acuerdo a Dean Kelley (1995)

El complemento de un lenguaje es la diferencia entre el mismo y su alfabeto. Para un lenguaje A y un alfabeto Σ , la expresión sería la siguiente:

$$\bar{A} = \Sigma^* - A$$

Concatenación y diferencia sobre un lenguaje: De acuerdo a Dean Kelley(1995) "la concatenación y la diferencia de lenguajes son incompatibles porque no se demuestra que $A(B-C) \neq AB-AC$."

Ejemplo de incumplimiento:

$$\Sigma = \{SO, S, OL, L\}$$

$$A = \{SO, S\}$$

$$B = \{OL\}$$

$$C = \{L\}$$

$$A(B - C) = A \cdot (\{OL\} - \{L\}) = \{SO, S\} \cdot \{OL\} = \{SOOL, SOL\}$$

$$AB - AC = \{SO, S\} \cdot \{OL\} - \{SO, S\} \cdot \{L\} = \{SOOL, SOL\} - \{SOL, SL\} = \{SOOL\}$$

$$\{SOOL, SOL\} \neq \{SOOL\}$$

Inverso del lenguaje:

De acuerdo a Dean Kelley (1995):

El inverso de un lenguaje es el lenguaje formado por las palabras del mismo invertidas. Un ejemplo sería que para un lenguaje $A = \{ \text{dog, bog} \}$, su inverso sería $A^1 = \{ \text{god, gob} \}$. La expresión es la siguiente:

$$A^I = \{ x^I \mid x \in A \}$$

Ejercicios de Ejemplo Resueltos

Ejercicio 1

Considere las palabras $u = a^2ba^3b^2$ y $v = bab^2$. Encuentre:

- uv y $|uv|$

$$uv = u \cdot v = aabaaabb \cdot babb = aabaaabbbabb$$

$$|uv| = |u| + |v| = 8 + 4 = 12$$

- vu y $|vu|$

$$vu = v \cdot u = babb \cdot aabaaabb = babbaabaaabb$$

$$|vu| = |v| + |u| = 4 + 8 = 12$$

- v^2 y $|v^2|$

$$v^2 = v \cdot v = babb + babb = babbabb$$

$$|v^2| = |v| + |v| = 4 + 4 = 8$$

Ejercicio 2

Suponga $u = a^2b$ y $v = b^3ab$. Encuentre:

- uvu

$$u \cdot v \cdot u = aab \cdot bbbab \cdot aab = aabbbbabaab$$

- λu

$$\lambda u = \lambda \cdot aab = aab$$

- $u\lambda$

$$u\lambda = aab \cdot \lambda = aab$$

- $u\lambda v$

$$u\lambda v = aab \cdot \lambda \cdot bbbab = aabbbbab$$

Ejercicio 3

Sea $w = abcd$. Encuentre todas las subpalabras de w

$$W = a^* b^* c^* d$$

$$W = ab^* c^* d$$

$$W = abc^* d$$

$$W = a^* b^* cd$$

$$W = a^* bcd$$

$$W = ab^* cd$$

$$W = a^* b^* c^* d^* \varepsilon$$

$$W = ab^* c^* d^* \varepsilon$$

$$W = abc^* d^* \varepsilon$$

$$W = a^* b^* cd^* \varepsilon$$

$$W = a^* bcd^* \varepsilon$$

$$W = ^* ab^* cd^* \varepsilon$$

$$W = \varepsilon^* a^* b^* c^* d$$

$$W = \varepsilon^* ab^* c^* d$$

$$W = \varepsilon * abc * d$$

$$W = \varepsilon * a * b * cd * \varepsilon$$

$$W = \varepsilon * a * bcd * \varepsilon$$

$$W = \varepsilon * ab * cd * \varepsilon$$

$$W = \varepsilon * a * b * c * d * \varepsilon$$

$$W = \varepsilon * ab * c * d * \varepsilon$$

$$W = \varepsilon * abc * d * \varepsilon$$

$$W = \varepsilon * a * b * cd * \varepsilon$$

$$W = \varepsilon * a * bcd * \varepsilon$$

$$W = \varepsilon * ab * cd * \varepsilon$$

Ejercicio 4

Sean $K = \{a, ab, a^2\}$ y $L = \{b^2, aba\}$ lenguajes sobre $A = \{a, b\}$. Encuentre:

- KL
- LL
- KK

Solución:

$$K \cdot L = \{a \cdot b^2, a \cdot aba, ab \cdot b^2, ab \cdot aba, a^2 \cdot b^2, a^2 \cdot aba\}$$

$$K \cdot L = \{abb, aaba, abbb, ababa, aabb, aaaba\}$$

$$L \cdot L = \{b^2 \cdot b^2, b^2 \cdot aba, aba \cdot b^2, aba \cdot aba\}$$

$$L \cdot L = \{b^4, b^2 \cdot aba, aba \cdot b^2, (aba)^2\}$$

$$L \cdot L = \{bbbb, bbaba, ababb, abaaba\}$$

$$K \cdot K = \{a \cdot a, a \cdot ab, a \cdot a^2, ab \cdot a, ab \cdot ab, ab \cdot a^2, a^2 \cdot a, a^2 \cdot ab, a^2 \cdot a^2\}$$

$$K \cdot K = \{a^2, a \cdot ab, a^3, ab \cdot a, (ab)^2, ab \cdot a^2, a^3, a^2 \cdot ab, a^4\}$$

$$K \cdot K = \{a^2, a \cdot ab, a^3, ab \cdot a, (ab)^2, ab \cdot a^2, a^2 \cdot ab, a^4\} \text{ (se saca el } a^3 \text{ de más)}$$

$$K \cdot K = \{aa, aab, aaa, aba, abab, abaa, aaab, aaaa\}$$

Ejercicio 5

Considere el lenguaje $L = \{ab, c\}$ sobre $A = \{a, b, c\}$. Encuentre:

- $L^0 = \{\lambda\}$
- $L^1 = \{ab, c\}$
- $L^2 = \{abab, abc, cab, cc\}$

Ejercicio 6

Sea $A = \{a, b, c\}$. Encuentre L^* , donde:

- A. $L = \{b^2\}$
- B. $L = \{a, b\}$
- C. $L = \{a, b, c^3\}$

A) $L = \{b^2\}$

$$L^* = \{\epsilon, b^2, b^4, b^6, \dots, b^{n \cdot 2}\}$$

B) $L = \{a, b\}$

$$L^* = \{ \epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, bba, bbb, \dots, L^n \}$$

$$C) \quad L = \{ a, b, c^3 \}$$

$$L^* = \{ \epsilon, a, b, c^3, aa, ab, ac^3, ba, bb, bc^3, c^3a, c^3b, c^6, \dots, L^n \}$$

Ejercicio 7

Dados los lenguajes $L = \{\text{bar, car, dog, elf}\}$ y $K = \{\text{bag, cat, dog, elf}\}$ sobre el alfabeto $A = \{ \lambda, a, b, c, d, e, f, g, l, o, r, t \}$ encuentre:

- [a] $L \cup K$
- [b] $L \cap K$
- [c] L^l
- [d] K^l
- [e] $L - K$

$$[a] \quad L \cup K = \{\text{bar, car, dog, elf, bag, cat}\}$$

$$[b] \quad L \cap K = \{\text{dog, elf}\}$$

$$[c] \quad L^l = \{\text{rab, rac, god, fle}\}$$

$$[d] \quad K^l = \{\text{gab, tac, god, fle}\}$$

$$[e] \quad L - K = \{\text{bar, car}\}$$

Ejercicio 8

Suponga dos lenguajes L_1 y L_2 . Si se sabe que $L_1 \cap L_2 = \emptyset$, ¿cuál es el resultado de $L_1 - L_2$?

Justifique.

Respuesta: $L1-L2=L1$

Justificación:

(cómo cito Kelley, 1995) Se dice que los conjuntos A y B son disjuntos si $A \cap B = \emptyset$

Siguiendo lo anterior L1 y L2 no tienen ningún elemento en común por lo que si a L1 le quitamos L2 (hacemos la diferencia) siendo que no L1 no tiene los elementos de L2, el lenguaje resultante es el mismo que L1

Ejercicio 9

Considere un alfabeto numerable $A = \{a_1, a_2, \dots\}$. Sea L_k el lenguaje sobre A que consta de las palabras w tales que la suma de los subíndices de las letras en w es igual a k.

(Por ejemplo, $w = a_2a_3a_3a_6a_4$ pertenece a L_{18} .)

- Encuentre todas las palabras de L_4 .
- Encuentre todas las palabras de L_6 .
- Encuentre la cantidad de palabras que componen a L_{12} .

Solución:

$L_4 = \{a_4, a_3a_1, a_1a_3, a_2a_2, a_2a_1a_1, a_1a_2a_1, a_1a_1a_2, a_1a_1a_1a_1\}$ 8 palabras

$L_6 = \{a_6, a_5a_1, a_1a_5, a_4a_2, a_2a_4, a_3a_3, a_4a_1a_1, a_1a_4a_1, a_1a_1a_4, a_3a_2a_1, a_3a_1a_2, a_2a_3a_1, a_2a_1a_3, a_1a_3a_2, a_1a_2a_3, a_2a_2a_2, a_3a_1a_1a_1, a_1a_3a_1a_1, a_1a_1a_3a_1, a_1a_1a_1a_3, a_2a_2a_1a_1, a_2a_1a_2a_1, a_2a_1a_1a_2, a_1a_1a_2a_2, a_1a_2a_1a_2, a_1a_2a_2a_1, a_2a_1a_1a_1a_1, a_1a_2a_1a_1a_1, a_1a_1a_2a_1a_1, a_1a_1a_1a_2a_1\}$

$a_1 a_1 a_1 a_1 a_2, a_1 a_1 a_1 a_1 a_1 a_1\}$ 32 palabras

- Si vemos la cantidad de palabras que se pueden formar para los primeros valores de k en L_k podemos ver que la cantidad de palabras siempre es el doble que en $k-1$:

$L_1 = 1 \text{ palabra}, L_2 = 2 \text{ palabras}, L_3 = 4 \text{ palabras}, L_4 = 8 \text{ palabras}, L_5 = 16 \text{ palabras},$
 $L_6 = 32 \text{ palabras}$

esto se podría representar con la fórmula 2^{k-1} , si introducimos $k = 12$ en esta fórmula, nos da como resultado $2^{12-1} = 2^{11} = 2048$ posibles palabras.

Gramáticas Formales

En esta sección del trabajo colaborativo vamos a desarrollar el tema de las "**Gramáticas Formales**". Exploramos en profundidad su definición y los componentes fundamentales que las conforman, como las variables no terminales, las variables terminales y las reglas de producción. Analizaremos cómo una cadena se deriva de otra y cómo se generan formas que pueden representarse a través de árboles de derivación. Además, abordaremos la importancia de las gramáticas en la generación de lenguajes, la equivalencia entre diferentes lenguajes, y estudiaremos la jerarquía de Chomsky, desglosando sus cuatro tipos de gramáticas y su impacto en el ámbito de la lingüística computacional.

Gramática: La gramática es un ente formal para especificar, de una manera finita, el conjunto de cadenas de símbolos que constituyen un lenguaje.

Definición Formal de Gramática

Una gramática es una cuádrupla: $G = (VT , VN , S , P)$ donde:

- $VT = \{\text{conjunto finito de símbolos terminales}\}$
- $VN = \{\text{conjunto finito de símbolos no terminales}\}$
- S = es el símbolo inicial y pertenece a VN .
- $P = \{\text{conjunto de producciones o de reglas de derivación}\}$

Todas las cadenas del lenguaje definido por la gramática están formadas con símbolos del vocabulario terminal **VT**.

El vocabulario no terminal **VN** es el conjunto de símbolos introducidos como elementos auxiliares para la definición de la gramática, y que no figuran en las sentencias del lenguaje

El símbolo inicial **S** es un símbolo no terminal a partir del cual se aplican las reglas de la gramática para obtener las distintas cadenas del lenguaje.

Las producciones **P** son las reglas que se aplican desde el símbolo inicial para obtener las cadenas del lenguaje. El conjunto de producciones P se define por medio de la enumeración de las distintas producciones, en forma de reglas o por medio de un metalenguaje por ejemplo **BNF** (Backus Naur Form) o **EBNF** (Extended Backus Naur Form).

La intersección entre el vocabulario terminal y no terminal es el conjunto vacío: $\{VN\} \cap \{VT\} =$

$\{\emptyset\}$

La unión entre el vocabulario terminal y no terminal es el vocabulario: $\{VN\} \cup \{VT\} = \{V\}$

Daremos un ejemplo:

Sea la gramática: $G = (VT, VN, S, P)$ donde $VT = \{a, b\}$, $VN = \{S\}$, y el conjunto de producciones es :

- $S \rightarrow aSb$
- $S \rightarrow ab$

Reglas de Producción

Sea Σ un alfabeto, llamamos producción (o regla) definida sobre ese alfabeto a un par ordenado de palabras (α, β) donde $\alpha, \beta \in \Sigma^*$. Se dice que α es la parte izquierda de la producción y que β es la parte derecha. A las producciones también se las llama reglas de derivación. Se representa $\alpha \rightarrow \beta$.

Derivación de Cadenas

La **derivación** de una cadena $w \in VT^*$ Se refiere a una secuencia de transformaciones en la que, empezando desde S , se aplican reglas de producción hasta obtener w . Cada paso de la derivación transforma una cadena intermedia aplicando una producción válida.

Derivemos la cadena $aabbbb$:

1. Comenzamos con el símbolo inicial S.

$$S \rightarrow aSb$$

2. Aplicamos nuevamente la regla $S \rightarrow aSb$ al símbolo S.

$$aSb \rightarrow aaSbb$$

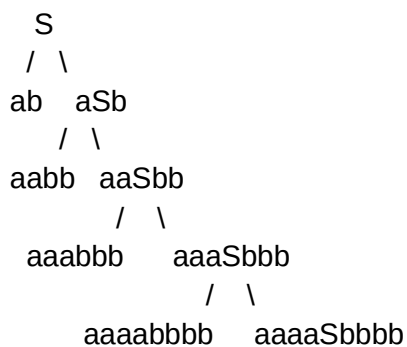
3. Ahora aplicamos la regla $S \rightarrow ab$ al símbolo S.

$$aaSbb \rightarrow aaabbbb$$

Fórmula Generada

La cadena generada es $a^n b^n$ donde $n > 0$ y además donde a y b siempre van a tener la misma cantidad de símbolos.

Árbol de derivación: Un árbol de derivación es una representación gráfica de la derivación de una forma sentencial



Como vemos siempre la cantidad de a y b serán las mismas y al tener el símbolo S podemos derivar siempre en $s \rightarrow aSb$, por lo que podríamos hacer un bucle infinito. Por esto queda correcta la notación $a^n b^n$.

Jerarquía de las Gramáticas

Chomsky definió cuatro tipos distintos de gramática en función de la forma de las reglas de derivación. La clasificación comienza con un tipo de gramática que pretende ser universal, aplicando restricciones a sus reglas de derivación se van obteniendo los otros tres tipos de gramáticas. Esta clasificación es jerárquica, es decir cada tipo de gramática engloba a todos los tipos siguientes.

Gramáticas de Tipo 0

También llamadas gramáticas no restringidas o gramáticas con estructura de frase. Las reglas de derivación son de la forma:

$$\alpha \rightarrow \beta$$

Ejemplo:

$$G = (VT, VN, S, P) \text{ donde } VT = \{a, b\}, VN = \{S, Y, Z\}$$

Producciones:

$$S \rightarrow aYb$$

$$Y \rightarrow aZb$$

$$Z \rightarrow aSb$$

$$S \rightarrow ab$$

Derivación ejemplo:

$$S \rightarrow aYb \longrightarrow aYb$$

$$Y \rightarrow aZb \longrightarrow aaZbb$$

$$Z \rightarrow aSb \longrightarrow aaaSbbb$$

$$S \rightarrow aYb \longrightarrow aaaaYbbbb$$

$$Y \rightarrow aZb \longrightarrow aaaaaYbbbbb$$

$$Z \rightarrow aSb \longrightarrow aaaaaaSbbbbbb$$

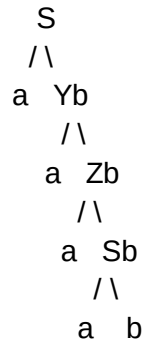
$$S \rightarrow ab \longrightarrow aaaaaaabbabbbb$$

Fórmula generada:

Este conjunto de reglas permite generar todas las cadenas de la forma $a^n b^n$

donde $n \geq 1$ lo que define el lenguaje anterior.

Árbol de derivación:



Gramáticas de Tipo 1

También llamadas gramáticas sensibles al contexto . En ellas las reglas de producción son de la forma:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

siendo $A \in VN$; $\alpha, \beta \in (VN \cup VT)^*$ $\gamma \in (VN \cup VT)^+$

Estas gramáticas se llaman sensibles al contexto, pues se puede reemplazar A por siempre que estén en el contexto $\alpha \dots \beta$

Ejemplo:

La gramática $G = (\{S, A, B\}, \{a, b\}, S, P)$ cuyas producciones P se muestran a continuación es de tipo 1.

Producción:

$$S \rightarrow aB$$

$$S \rightarrow bA$$

$$A \rightarrow a$$

$$A \rightarrow aS$$

$$A \rightarrow bAA$$

$$B \rightarrow b$$

$$B \rightarrow bS$$

$$B \rightarrow aBB$$

Derivación ejemplo:

$$S \rightarrow bA \rightarrow bbAA$$

$$bbAA \rightarrow bbaaS$$

$$bbaaS \rightarrow bbaaaB$$

$$bbaaaB \rightarrow bbaaab$$

Fórmula generada:

Este conjunto de reglas permite generar todas las cadenas de la forma $\alpha A \beta \rightarrow \alpha \gamma \beta$

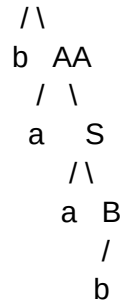
donde A es un símbolo no terminal, $\alpha \gamma \beta$ son cadenas de terminales y/o no terminales, por último γ es una cadena que es al menos tan larga como A.

$$L(G) = \{w \mid w \text{ es una cadena de la forma } b^m a^n b^p \text{ con } m, n, p \geq 1\}$$

La gramática genera cadenas que comienzan con una o más b's, seguidas de una o más a's y luego seguidas de una o más b's.

Árbol de derivación para bbaaab:

$$\begin{array}{c} S \\ / \quad \backslash \\ b \quad A \end{array}$$



Gramáticas de Tipo 2

Las gramáticas de tipo 2 también se denominan gramáticas de contexto libre o libres de contexto. Son de un interés muy particular, ya que las mismas definen las reglas sintácticas de la mayoría de los lenguajes de programación. Sus reglas de producción tan sólo admiten tener un símbolo no terminal en su parte izquierda, es decir son de la forma:

$$A \rightarrow \alpha, \text{ siendo } A \in VN \text{ y } \alpha \in (VN \cup VT)^+$$

Si cada regla se representa como un par ordenado (A, α) , el conjunto P es un subconjunto del conjunto producto cartesiano $VN \times \{VN \cup VT\}^+$, es decir

$$P \subset \{N \times (\{VN\} \cup \{VT\})^+\}$$

La denominación contexto libre se debe a que se puede cambiar A por α , independientemente del contexto en que aparezca A .

La gramática $G = (\{A, C, B\}, \{0, 1\}, A, P)$ cuyas producciones P se muestran a continuación es de tipo 2.

Producción:

$$A \rightarrow BC$$

$$B \rightarrow 01$$

$$B \rightarrow 0B1$$

$$C \rightarrow 2$$

$$C \rightarrow 2C$$

Derivación ejemplo:

$$A \rightarrow BC$$

$$BC \rightarrow 01C$$

$$01C \rightarrow 012$$

Fórmula generada:

En esta vemos que las producciones de **B** permiten generar $0^n 1^n$ donde $n \geq 1$

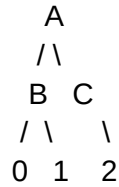
En las producciones de **C** vemos que 2^n donde $n \geq 1$ Y en las de **A** vemos que

podemos combinar las producciones B y C para generar $0^n 1^n 2^m$ donde $n \geq 1$ y $m \geq 1$,

por lo que su lenguaje y notación sería

$$L(G) = \{w \mid w = 0^n 1^n 2^m \text{ con } n \geq 1 \text{ y } m \geq 1\}$$

Árbol de Derivación para 012



Posibles cadenas generadas por la gramática

Algunas otras cadenas que pueden generarse mediante esta gramática incluyen:

- 00112: Aplicando la producción $B \rightarrow 0B1$ una vez, obtenemos $0^2 1^2$ para B, y luego $c \rightarrow 2C$, obtenemos 2^2
- 000111222: Aplicando la producción $B \rightarrow 0B1$ dos veces, obtenemos $0^3 1^3$ para B y aplicando $c \rightarrow 2C$ dos veces, obtenemos 2^3

Conclusión

Las gramáticas de tipo 2 son la base para definir y analizar la sintaxis de los lenguajes de programación. Al ser libres de contexto, permiten describir las estructuras del lenguaje de manera clara, sin depender del entorno, lo que facilita el diseño de compiladores y el análisis de programas. Por eso, los lenguajes de programación se basan en este tipo de gramáticas para definir cómo deben estructurarse sus construcciones sintácticas.

Gramáticas de Tipo 3

Las gramáticas de tipo 3, también denominadas regulares o gramáticas lineales a la derecha, son las más precisas y comienzan sus reglas de producción por un símbolo terminal, que puede ser seguido o no por un símbolo no terminal, es decir son de la forma:

$$A \rightarrow aB$$

$$A \rightarrow a$$

donde $A, B \in VN$ y $\alpha \in VT$

La gramática $G = (\{a,b\}, \{A,S\}, S, P)$ donde P son las producciones que se muestran a continuación es de tipo 3.

Producciones:

$$S \rightarrow aS$$

$$S \rightarrow aA$$

$$A \rightarrow bA$$

$$A \rightarrow$$

Derivación ejemplo:

$$S \rightarrow aA$$

$$aA \rightarrow abA$$

$$abA \rightarrow abbA$$

$$abbA \rightarrow abbb$$

Fórmula generada:

El lenguaje generado por esta gramática es:

$$L(G) = \{w \mid w = a^n b^m \text{ con } n \geq 1 \text{ y } m \geq 1\}$$

Árbol de Derivación para abbb:

S

```

  /\
a  A
  /\
b  A
  /\
b  A
  /\
b  ε

```

Gramáticas equivalentes

Se puede demostrar que si todas las reglas de una gramática cumplen la condición de no decrecimiento, se puede hallar una gramática equivalente con las producciones sensibles al contexto. Esta segunda propiedad combinada con la primera hace que se pueda intercambiar la característica de no decrecimiento con la definición. Veremos un ejemplo:

Sea la gramática $G = (\{S, B, C\}, \{a, b, c\}, S, P)$ donde P son las producciones:

$S \rightarrow aSBC$

$S \rightarrow aBC$

$CB \rightarrow BC$

$bB \rightarrow bb$

$bC \rightarrow bc$

$cC \rightarrow cc$

$aB \rightarrow ab$

La gramática anterior no es de tipo 1 según la definición dada, ya que la regla $CB \rightarrow BC$ no respeta el contexto. Sin embargo puede apreciarse que todas las reglas de esta gramática son no decrecientes, por lo tanto es posible encontrar una gramática equivalente que genere el mismo lenguaje. Se puede sustituir la regla $CB \rightarrow BC$ por :

$$CB \rightarrow XB$$

$$XB \rightarrow XY$$

$$XY \rightarrow BY$$

$$BY \rightarrow BC$$

Puede observarse que ambas gramáticas son equivalentes y que generan el lenguaje:

$$L(G) = \{ a^n b^n c^n / n \geq 1 \}$$

Gramáticas en Lenguajes de Programación

En cuanto a los lenguajes de programación, mayormente se utilizan las gramáticas de tipo 2 ya que la principal ventaja radica en su flexibilidad, no imponen grandes restricciones como otros tipos de gramáticas. Algunos de los principales casos de esto son:

1. **Expresiones aritméticas:** Hay operaciones con paréntesis las cuales contienen signos de operación y los valores que se están operando o las variables que contienen dichos valores, esto es posible con las gramáticas libres de contexto (tipo 2) ya que permiten guardar la

procedencia de operadores y sus asociaciones. Las gramáticas regulares (tipo 3) no permiten manejar la anidación con paréntesis.

2. **Bloques de código:** Hay anidaciones de bloques de código, esto es posible en las gramáticas libres de contexto ya que permite reconocer la estructura jerárquica de los bloques de texto. Las gramáticas regulares no permiten manejar la anidación de bloques de forma adecuada.
3. **Declaraciones de variables:** Hay declaración de variables con distintos tipos de datos y pueden ser anidadas dentro de bloques de código, esto es posible en las gramáticas libres de tipo 2 ya que permiten especificar distintas categorías de símbolos terminales. Las gramáticas de tipo 3 son menos flexibles en cuanto a la definición de símbolos.
4. **Funciones:** Hay retorno de un valor con un tipo específico, uno o varios parámetros de distintos tipos, y dentro del cuerpo de la función se pueden contener varias declaraciones, incluyendo declaración de variables y llamados a otras funciones. Esto es posible en las gramáticas libres de contexto ya que permite describir la lista de parámetros de manera flexible, describir la jerarquía de las expresiones y especificar el tipo de retorno de una función. Las gramáticas regulares no pueden expresar recursión ni tampoco capturar la estructura anidada del cuerpo de una función.

Como mencionaron John Hopcroft, Rajeev Motwani, Jeffrey Ullman, (2006):

Las gramáticas son modelos útiles en el diseño de software que sirve para procesar datos con una estructura recursiva. El ejemplo más conocido es el de un “analizador sintáctico” (parser), el componente de un compilador que se ocupa de las funciones anidadas recursivamente de los lenguajes de programación típicos, tales como expresiones aritméticas, condicionales, etc. (p. 4)

Esto permite el uso de las expresiones aritméticas y las funciones. El reconocimiento de la jerarquía de los bloques de código son abarcables gracias a los árboles de derivación, como afirman Hopcroft et al. (2006):

Lo más importante es que el árbol, conocido como “árbol de derivación”, cuando se emplea en un compilador, es la estructura de datos que representa el programa fuente. En un compilador, la estructura del árbol del programa fuente facilita la traducción del programa fuente a código ejecutable permitiendo que el proceso de traducción sea realizado por funciones naturales recursivas. (p. 154)

También describen que “Los árboles son colecciones de nodos, que mantienen una relación padre-hijo. Un nodo tiene como máximo un padre, que se dibuja por encima del mismo, y cero o más hijos, que se dibujan por debajo” (p. 155). Esto permite la creación de bloques de código en estructuras jerárquicas.

La declaración de variables, su almacenamiento y reconocimiento de alcance en el bloque de código en el que se encuentre es posible gracias al uso de autómatas a pila, tal como dijeron John Hopcroft, Rajeev Motwani, Jeffrey Ullman, (2006)

El autómata a pila es un autómata finito no determinista con transiciones- ϵ y una capacidad adicional: una pila en la que se puede almacenar una cadena de 'símbolos de pila'. . . . los autómatas a pila reconocen todos los lenguajes independientes del contexto y sólo estos. (p. 187)

Dos ejemplos donde se emplea el uso de la gramática de tipo 2 son los siguientes:

1. En la mayoría de los lenguajes de programación los cuatro operadores aritméticos (suma, resta, multiplicación y división), son asociativos por la izquierda. Por ejemplo decimos que el operador + se asocia por izquierda porque un operando con signos positivos a ambos lados de él pertenece al operador que está su izquierda: Ej: $A + B$

+ 5, tenemos + B+, el operando B con signo positivos a ambos lados pertenece al operador que está a su izquierda, es decir, primero se realiza $A + B$ y luego se suma 5. Se podría representar: $(A+B) + 5$. (Universidad Tecnológica Nacional, 2014, p. 50).

2. Una regla gramatical como $E \Rightarrow E + E$ establece que una expresión puede formarse tomando cualesquiera dos expresiones y conectándose mediante un signo más; esta regla es típica de cómo se forman las expresiones en los lenguajes reales de programación." (John Hopcroft, Rajeev Motwani, Jeffrey Ullman, 2006, p. 4)

Por otro lado las gramáticas de tipo 3, son el resultado de aplicar la mayor cantidad de restricciones a las reglas de producción. Como consecuencia de esto, los lenguajes generados son menos expresivos. A pesar de esta limitación, este tipo de gramática se sigue utilizando en la programación debido a su enorme capacidad para definir expresiones regulares. Estas son fundamentales en diversas aplicaciones, como por ejemplo en el reconocimiento de patrones, donde la simplicidad y la eficiencia de los autómatas finitos es ideal.

Ejemplos:

1. La expresión regular estilo UNIX `'[A-Z][a-z]*[ ][A-Z][A-Z]'` representa palabras que comienzan por una letra mayúscula seguida de un espacio y de dos letras mayúsculas. Esta expresión representa patrones de texto que podrían corresponderse con el nombre de una ciudad y un estado. (John Hopcroft, Rajeev Motwani, Jeffrey Ullman, 2006, p. 4)
2. La gramática regular para un número decimal simple puede definirse mediante una expresión regular que permite una secuencia de dígitos, con la opción de tener un punto decimal seguido por más dígitos. La estructura básica de esta gramática es

lineal y no admite anidamiento o estructuras complejas.

Para ilustrar, consideremos la expresión regular que valida números decimales en una cadena. La gramática puede ser representada como sigue:

Regla de producción:

$$S \rightarrow D + (.D+)?$$

Donde S es el símbolo inicial, D representa un dígito (0-9), y (.D+)? indica que el punto decimal y los dígitos posteriores son opcionales.

En conclusión, las gramáticas libres de contexto se emplean en la mayoría de los lenguajes de programación, mientras que las gramáticas regulares, las cuales son más “limitadas” cuentan con mayor utilidad en aplicaciones específicas como el procesamiento de patrones simples.

Teoría de autómatas

En las anteriores unidades estuvimos viendo que un lenguaje se conforma mediante un conjunto de diversos elementos de un solo alfabeto. Por lo tanto, las posibilidades de combinación de los mismos son infinitas, lo que nos llevaría a que un lenguaje no tenga un conjunto finito de palabras que sean válidas en él. Aquí es donde aparecen los autómatas, los cuales permiten determinar si una palabra corresponde o no al lenguaje en cuestión.

Los autómatas finitos o máquinas de estado finito son un conjunto de transiciones que nos permiten ver cómo pasa de un estado a otro, y esa cantidad de estados es un conjunto numerable (finitos) que se mantienen constantes en el tiempo. Se aplican para el reconocimiento de palabras de lenguajes de tipo 3.

Además, los autómatas son fundamentales en el estudio de lenguajes formales, ya que permiten modelar cómo las máquinas procesan cadenas de símbolos. Su aplicación no solo se limita al

reconocimiento de lenguajes regulares, sino también en áreas como el diseño de circuitos digitales y la programación de sistemas embebidos. Este enfoque es clave para entender los lenguajes regulares y su relación con las gramáticas de tipo 3 dentro de la jerarquía de Chomsky, un concepto central en la teoría de lenguajes y computación (Hopcroft, Motwani, & Ullman, 2002).

Componentes

$$A = (Q, V, \delta, q_0, F)$$

A= Autómata

Q = Conjunto de estados

V = Alfabeto (vocabulario)

δ = funciones de transición (como se hace para cambiar de estado)

q_0 = Estado inicial

F = estado final

Se puede definir de dos formas, ambas son análogas y podemos ir de una a la otra: tabla de transiciones y diagrama de estados.

Tabla de transiciones

	X	Y
$\rightarrow A$	C	B
B	C	--
<u>C</u>	--	B

Figura 1. Ejemplo de una tabla de transiciones (elaboración propia)

Una tabla de transición es una representación tabular de los cambios de estado que ocurren en un autómata finito o en cualquier otro sistema basado en estados (como una máquina de estados finitos o un autómata de pila), en respuesta a una serie de entradas. Sirve para describir cómo el sistema pasa de un estado a otro según las entradas recibidas.

La tabla de transición es una herramienta fundamental para analizar y diseñar autómatas. A partir de esta representación, es posible determinar las reglas exactas de transición entre estados, facilitando el entendimiento de lenguajes formales y computación. En particular, permite visualizar de manera clara los resultados de las interacciones entre entradas y estados, proporcionando un método eficiente para el diseño y verificación de sistemas (Hopcroft, Motwani, & Ullman, 2002).

Componentes

Estados: Los posibles estados en los que puede estar el sistema.

Entradas: Los símbolos o valores que causan cambios de estado.

Transiciones: El nuevo estado resultante cuando el autómata recibe una entrada específica estando en un estado determinado

Diagrama de estados

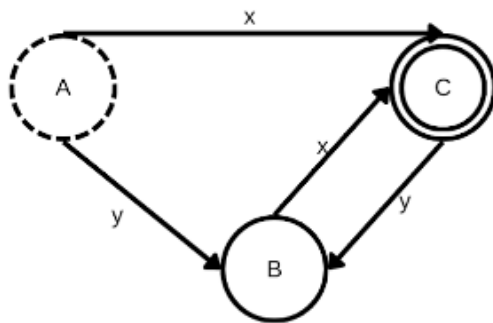


Figura 2. Ejemplo de un diagrama de estados (elaboración propia)

Un diagrama de estado es una representación gráfica de cómo un sistema basado en estados (como una máquina de estados finitos o un autómatas) evoluciona en respuesta a diferentes eventos o entradas. Cada estado en el que puede estar el sistema se representa como un nodo, y las transiciones entre los estados se representan con flechas dirigidas que indican cómo el sistema cambia de estado en función de una entrada específica, el gráfico indica cual es el estado inicial y final

Nos permiten identificar si algunas palabras pertenecen o no al lenguaje, una palabra pertenece al lenguaje si se condice con las transiciones y termine en un estado final.

Según la teoría clásica de los autómatas, los diagramas de estado son cruciales para entender el comportamiento de las máquinas de estado finito. Estas máquinas pueden ser modeladas a través de autómatas deterministas o no deterministas, los cuales aceptan lenguajes regulares. La representación de estados y transiciones es esencial para formalizar y analizar problemas computacionales, tanto en teoría como en aplicaciones prácticas (Hopcroft, Motwani, & Ullman, 2002).

Para que se cumpla la unicidad

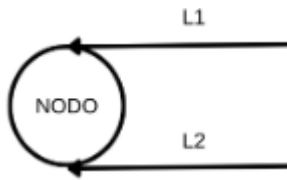


Figura 3. Condición que se debe cumplir para que exista unicidad (elaboración propia)

A un nodo no puedo llegar con la misma letra más de una vez

Tipos de autómatas

Determinísticos

- No tienen que haber λ en las transiciones
- Delta es en transición que cumpla la unicidad

No determinísticos

- Si se realiza transiciones con λ
- No se cumple la unicidad en delta

Si bien existen las dos condiciones, vamos a tomar en la cursada que la unicidad se cumple cuando a un nodo ingreso solamente por una letra

Dado un autómata existe un único lenguaje, pero un lenguaje puede tener asociado varios autómatas

Reglas Chomsky



Figura 4. Primera regla de Chomsky (elaboración propia)

2-

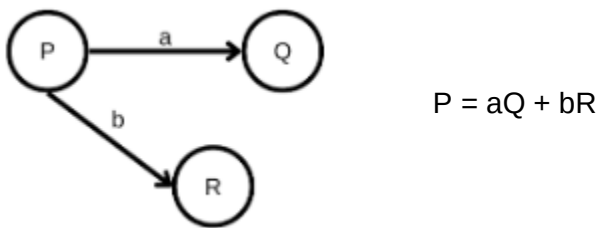


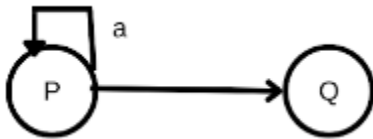
Figura 5. Segunda regla de Chomsky (elaboración propia)

3-



Figura 6. Tercera regla de Chomsky (elaboración propia)

4-



$$P = aP + bQ \rightarrow P = a^*. (bQ)$$

Figura 7. Cuarta regla de
(elaboración propia)

Chomsky

5-



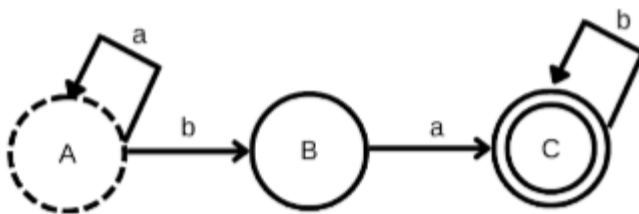
$$P = \lambda$$

Figura 8. Quinta regla de Chomsky (elaboración propia)

Forma de relacionar el autómata con lenguajes y gramáticas

Autómata $\square$ Lenguaje

1-



$$L = a^* b a b^*$$

Figura 9. Ejemplo de un Autómata (elaboración propia)

2-

	0	1
$\rightarrow A$	b	a
<u>B</u>	b	b

$$L = 1^* 0 (0 1)^*$$

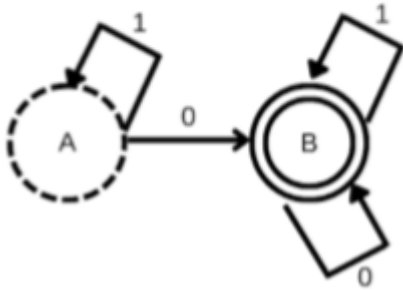


Figura 10. Transformación de tabla de transición a diagrama de estados (elaboración propia)

Lenguaje $\square$ Autómata

$$L = \{(ab)^m ca^p\}$$

$$m \geq 0$$

$$p > 0$$

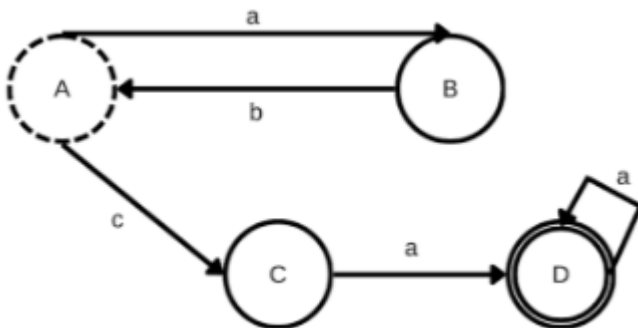


Figura 11. Ejemplo de diagrama de estados (elaboración propia)

Autómata $\square$ Gramática

$$A = (Q, V, \delta, q_0, F)$$

$$G = (V_n, V_t, P, S)$$

$$V_n = Q \quad V_t = V \quad S = q_0$$

P = Reglas de producción $\square$ vínculo entre las transiciones y el estado

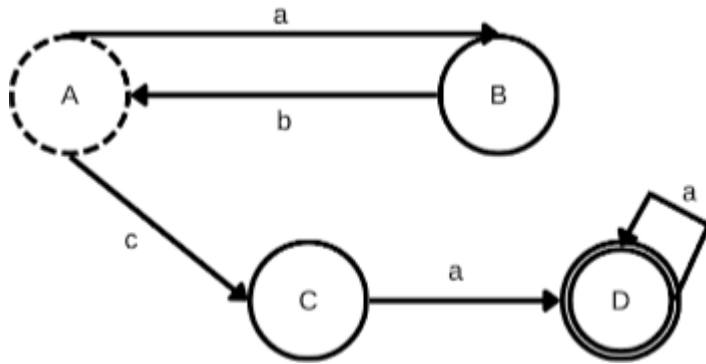


Figura 12. Ejemplo de diagrama de estados (elaboración propia)

$$G = (\{A, B, C, D\}; \{0,1,2\}; P; A)$$

$$\left. \begin{array}{l} A \square 0A + 2B \\ B \square 1D + 0C \\ C \square 0C + 1D + \lambda \\ D \square 0D + 2C \end{array} \right\} P$$

Gramática $\square$ Autómata

$$G = (\{A, B, C, D\}; \{0,1,2\}; P; A)$$

$$\left. \begin{array}{l} A \square 1B + 0A \\ B \square 0C + 0 \\ C \square 2C + 2 \end{array} \right\} P$$

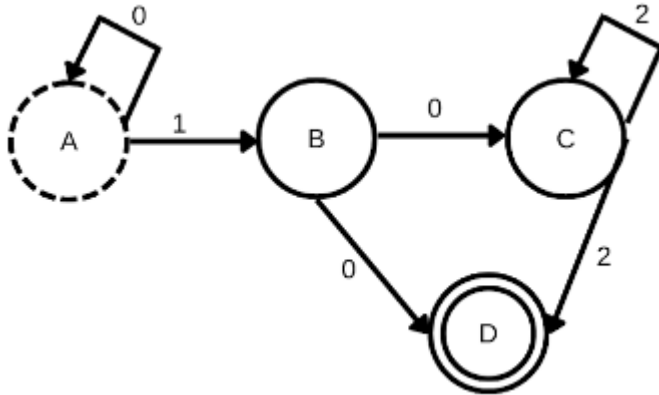


Figura 13. Ejemplo de diagrama de estados (elaboración propia)

Resolución ejercicio

Con la siguiente tabla de decisión dibujar el diagrama del autómata, indicar si es determinístico y construir la gramática regular del lenguaje.

	0	1	2
a	a	b	d
b	a	b	c
c	c	d	a
d	c	c	a

Figura 14. Ejercicio a resolver (extraído del campus)

Con la tabla podemos armar el diagrama de transición

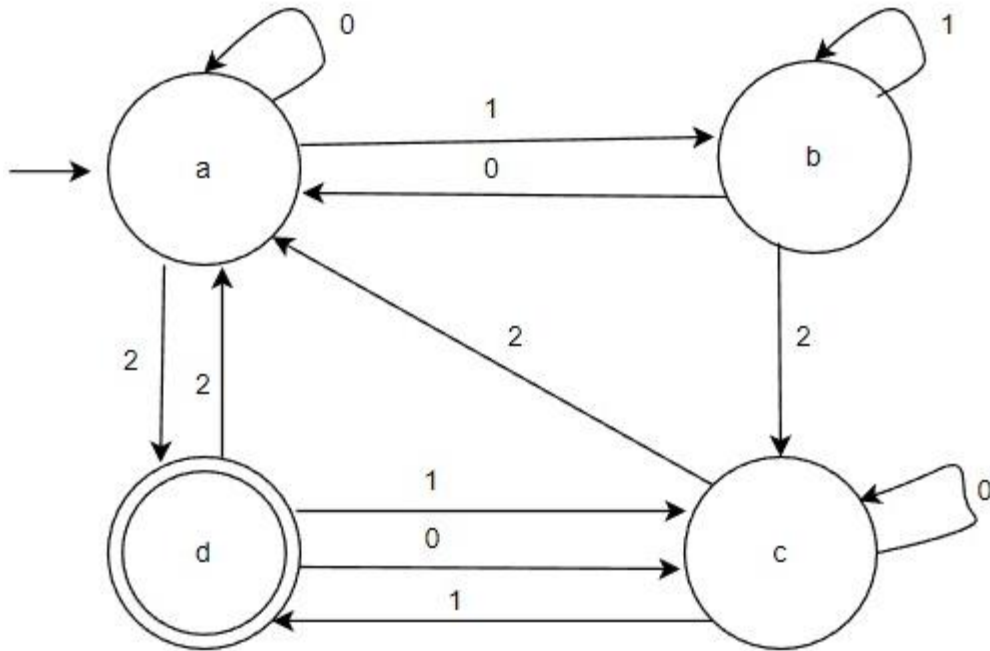


Figura 15. Ejemplo de diagrama de estados (elaboración propia)

Gramática

$A1 = (\{A, B, C, D\}; \{0, 1, 2\}; S; A; \{D\})$

Estados: A, B, C, D

Entradas: 0, 1, 2

Estado inicial: A

Estado Final: D

$S \rightarrow$ Transiciones: $A=0^* + B.1 + 2.D$

$B=1^* + 2.C + 0.A$

$C=0^* + 2.A + 1.D$

$D=(0,1).c + 2.A$

Basándonos en la primer regla del criterio de unicidad, podemos decir que es determinístico ya que para cada nodo una misma entrada no puede hacer más de 1 transición

Resolución ejercicio de aceptación de expresión regular

Ejercicio A:

Dado el alfabeto

$$\Sigma = \{0,1\}$$

construir un **Autómata Finito Determinista** de 4 estados como máximo, que acepte el lenguaje representado por la siguiente expresión regular

$$((01 + 10)(11)^*0)^*(01 + 10)(11)^*$$

Figura 16. Enunciado del ejercicio (extraída del documento “Prácticas de Parcial”)

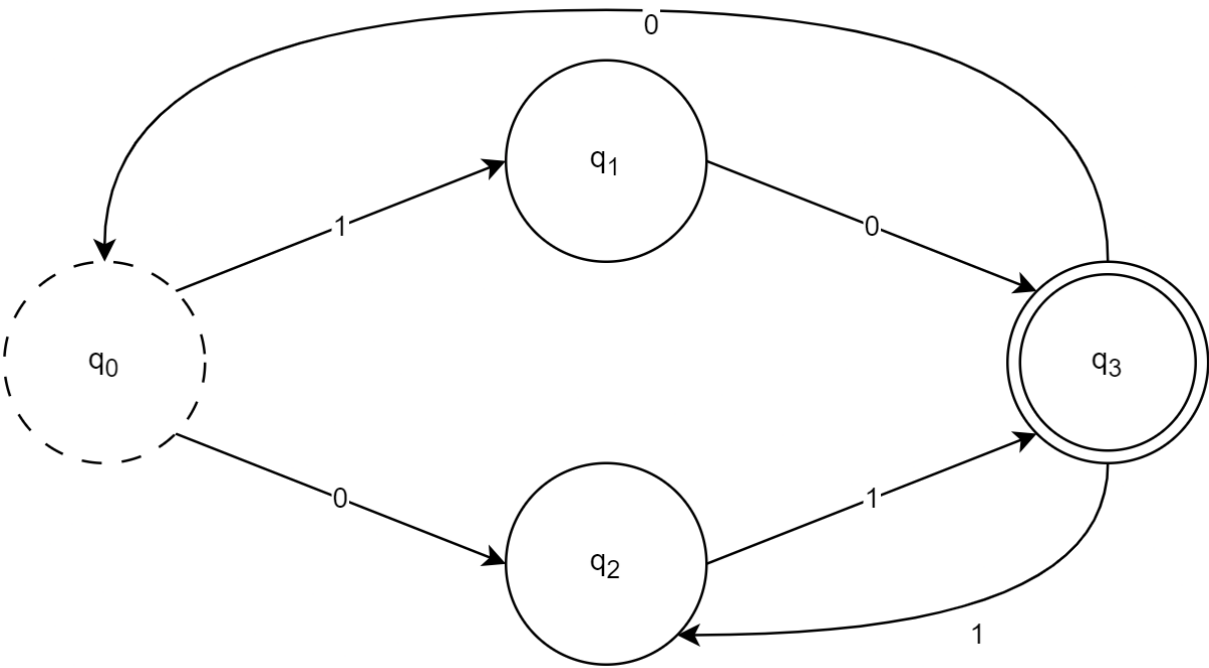


Figura 17. Resolución del enunciado (elaboración propia)

	0	1
q ₀	q ₂	q ₁
q ₁	q ₃	-
q ₂	-	q ₃
q ₃	q ₀	q ₂

Figura 18. Tabla de transiciones relacionada con el enunciado (elaboración propia)

Para resolver este ejercicio, lo que hicimos fue identificar los distintos componentes de la expresión regular del lenguaje con el objetivo de pensar cada componente por separado intentando aplicar las reglas de Chomsky y lograr conectarlas siempre respetando el criterio de la unicidad.

A partir de ese análisis identificamos los siguientes componentes:

- $(01 + 10)$
- $(11)^*$
- 0

y las siguientes conclusiones:

- $((01 + 10)(11)^*0)^*$ nos indica que todo el conjunto puede estar cero o más veces, por lo tanto, debemos diseñar un diagrama de transiciones que permita cumplir esa condición.
- $(11)^*$ nos indica que esa combinación de símbolos puede estar cero o más veces y que siempre debe darse así, en pares de unos.
- $((01 + 10)(11)^*0)^*$ es casi igual que $(01 + 10)(11)^*$, por lo que nos da indicios de que el diagrama de transiciones debe seguir algún esquema de ciclo. Sin embargo, también advertimos que $(01 + 10)(11)^*$ sí se debe cumplir obligatoriamente.
- La palabra mínima que admite este autómata es 01 o 10, según el camino que se decida tomar, ya que $((01 + 10)(11)^*0)^*$ es totalmente opcional como indicamos anteriormente, mientras que $(01 + 10)(11)^*$ tiene como opcional el $(11)^*$.
- La palabra máxima que admite este autómata no existe ya que contiene secciones que se pueden repetir indefinidamente como dijimos anteriormente.

Con toda esta información reunida, procedimos al armado del diagrama de estados y la tabla de transiciones visualizada anteriormente.

Eficiencia de Algoritmos

En el desarrollo de algoritmos y estructuras de datos, la eficiencia es un factor crítico que determina la viabilidad de las soluciones a problemas computacionales. La eficiencia de un algoritmo se refiere al consumo de algún recurso y normalmente es una medida de cómo el recurso es consumido. Lo que se pretende es tener instrumentos de análisis que nos permitan comparar algoritmos y elegir el más eficiente.

La complejidad algorítmica es una medida que permite evaluar el rendimiento de un algoritmo en términos de tiempo y espacio, proporcionando una perspectiva sobre cómo se comportará a medida que aumenta el tamaño de la entrada.

Complejidad Espacial

La complejidad espacial refiere al almacenamiento requerido por el algoritmo, y puede medirse por la cantidad de memoria usada para almacenar objetos tales como variables, constantes, arreglos.

Complejidad Temporal

La complejidad temporal refiere al tiempo que toma un algoritmo en ser ejecutado, y se mide contando el número de instrucciones a ser ejecutadas con una entrada de tamaño n .

Existen dos formas de medir la complejidad de un algoritmo en base al análisis temporal de los mismos: el análisis empírico y el análisis teórico.

Análisis Empírico

En este se calcula cuánto tiempo tarda un algoritmo en resolver un problema para distintos tamaños de entrada. Para llevarlo a cabo se deben seguir los siguiente pasos:

Paso 1. Implementar el algoritmo.

Figura 19

Algoritmo de Ejemplo

```
def sum_n(n: int) -> int:
    """returns the sum of the first n numbers"""

    if not isinstance(n, int) or n < 0:
        return None

    result = 0
    for i in range(n+1):
        result += i
    return result
```

Nota. Se muestra un ejemplo de un algoritmo que devuelve la suma de los primeros n números.

Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.5) por Luciano Moliterno (2025).

Paso 2. Incluir instrucciones que permitan medir el tiempo de ejecución.

Figura 20

Agregado de Instrucciones de Medición del Tiempo en el Algoritmo

```
import time

def sum_n(n: int) -> (int, float):
    start = time.time() # returns the time in milliseconds

    if not isinstance(n, int) or n < 0:
        return None

    result = 0

    for i in range(n+1):
        result += i
    end = time.time()
    return result, end-start # we also return the time
```

Nota. Se muestra el agregado de instrucciones para la medición de tiempo en el algoritmo de ejemplo. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.5) por Luciano Moliterno (2025).

Paso 3. Ejecutar el programa con entradas de diferentes tamaños y obtener los tiempos de ejecución para cada tamaño. Como existe una variabilidad en los tiempos que se puede tardar con

el mismo tamaño de entrada, se hace la misma prueba varias veces y se toma el valor medio de estas.

Figura 21

Ejecución del Programa para Tamaños Diferentes

```
if __name__ == '__main__':
    for i in range(1, 8):
        input_size = pow(10, i)
        result_sum, execution_time = sum_n(input_size)
        print("sumN({})={}, time={} sg".format(input_size, result_sum, execution_time*100))

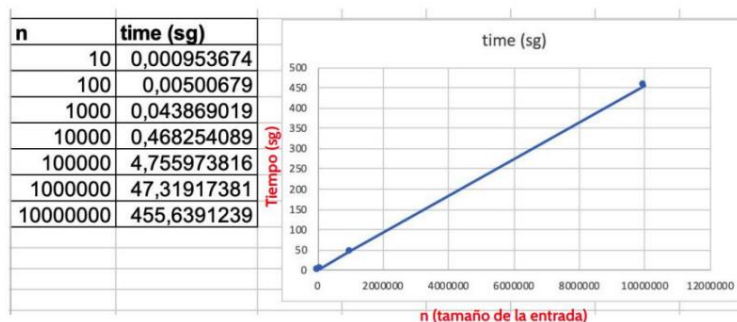
sumN(10)=55, time=0.00095367431640625 sg
sumN(100)=5050, time=0.0040531158447265625 sg
sumN(1000)=500500, time=0.04291534423828125 sg
sumN(10000)=50005000, time=0.45299530029296875 sg
sumN(100000)=5000050000, time=4.713773727416992 sg
sumN(1000000)=500000500000, time=49.21984672546387 sg
sumN(10000000)=50000005000000, time=456.09593391418457 sg
```

Nota. Se muestra el resultado de las ejecuciones del algoritmo de ejemplo con diferentes tamaños de entrada. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.6) por Luciano Moliterno (2025).

Paso 4. Importar los resultados a un fichero excel y mostrarlos en un gráfico de dispersión.

Figura 22

Gráfico del análisis empírico



Nota. El gráfico representa la dispersión en base a los resultados de las ejecuciones del algoritmo de ejemplo con diferentes tamaños de entrada. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.6) por Luciano Moliterno (2025).

Pero el análisis empírico tiene un gran problema, que depende de la máquina donde esté ejecutando el algoritmo y del lenguaje en que esté escrito, es decir, las pruebas empíricas están

totalmente condicionadas por el software, hardware, y lenguaje. Es por esto que es importante detallar sobre qué máquina, lenguaje, y tecnología se están ejecutando las pruebas.

Analisis Teorico

A diferencia del análisis empírico, en el análisis teórico no hace falta implementar el algoritmo, este consiste en obtener la función $T(n)$, que representa el número de operaciones que deben ser ejecutadas para una entrada de tamaño n . Gracias a ser una función, es posible considerar todas las posibles entradas, e independizarse del entorno del hardware y software.

Para esto es necesario identificar las acciones atómicas. Estas son aquellas que ocupan un tiempo de máquina, son acciones que a la máquina le llevan una unidad de su tiempo de medida. A cada instrucción del algoritmo se la va a separar en operaciones atómicas, y como un algoritmo va a estar compuesto de muchas instrucciones, se van a ir contando los tiempos de máquina que demanda cada instrucción para saber cuánto tiempo tarda el algoritmo completo. A continuación se muestra una tabla con los tiempos de máquina que ocupa cada operación:

Tabla 1

Tiempo de máquina usado por cada operación

Operaciones	Tiempo de Máquina
Operaciones Primitivas (operaciones aritméticas, operaciones lógicas, asignar un valor a una variable, acceder a una posición de memoria, escribir o leer un valor variable).	$T(n) = 1$
Operaciones selectivas o condicionales (if, else).	$T(n) = \max(TB1(n), TB2(n) \dots TBK(n))$ Donde $T(n)$ es el máximo T de los bloques que se pueden ejecutar. Y $TBK(n)$ representa el número de operaciones correspondiente al bloque de instrucciones dentro de cada condición.
Operaciones Bucles (for, while).	$T(n) = TB(n) * \text{rep. del bucle} \quad \text{cant de bucles}$ Donde $TB(n)$ es el número de operaciones correspondiente al bloque de instrucciones dentro del bucle.

Nota. La tabla muestra el tiempo de máquina que ocupa cada operación en particular.

Para ejemplificar, se presentan dos algoritmos que realizan la suma de los primeros n números, pero uno lo hace aplicando un bucle, y otro lo hace mediante Gauss.

Figura 23

Algoritmo que suma los primeros n números

```
def sum_n(n: int) -> int:
    """returns the sum of the first n numbers"""
    result = 0          # 1
    for j in range(n+1): # loop will be executed n+1 times
        result += j     # 2
    return result      # 1
```

Nota. La imagen muestra un algoritmo que suma los primeros n números usando un bucle. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.17) por Luciano Moliterno (2025).

Para calcular $T(n)$ para el algoritmo de la figura 23, tendremos que hacer el siguiente cálculo:

$$T(n) = 1 + 2 * (n + 1) + 1 = 2n + 4$$

Figura 24

Algoritmo que suma los primeros n números aplicando Gauss

```
def sum_gauss(n: int) -> int:
    """returns the sum of the first n numbers, by
    applying Gauss sum"""
    return n*(n+1) // 2
```

Nota. La imagen muestra un algoritmo que suma los primeros n números usando Gauss. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.17) por Luciano Moliterno (2025).

Para calcular $T(n)$ para el algoritmo de la figura 24, tendremos que hacer el siguiente cálculo:

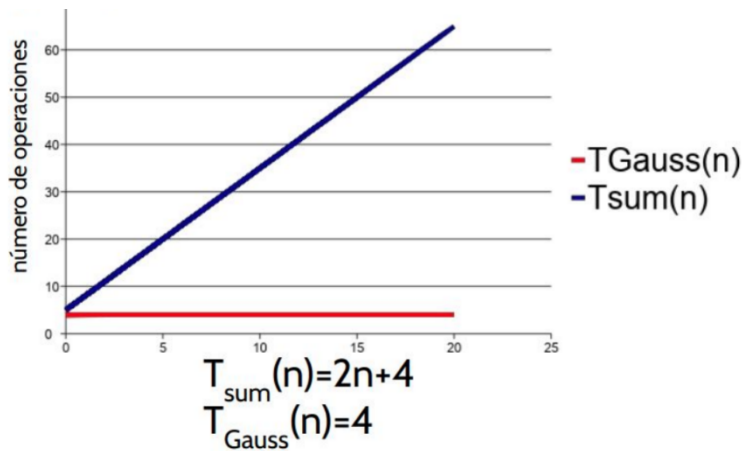
$$T(n) = 1(\text{suma}) + 1(\text{multiplicación}) + 1(\text{división}) + 1(\text{return}) = 4$$

Como se puede ver, aunque ambos algoritmos realicen el mismo cálculo, el algoritmo de la figura 23 depende de n , pero el algoritmo de la figura 24 no. Esto nos indica que el segundo

algoritmo es más eficiente que el primero ya que siempre tomará el mismo tiempo, esto lo podemos ver representado en el siguiente gráfico:

Figura 25

Comparación de las funciones de cada algoritmo



Nota. El gráfico representa la comparativa de las funciones temporales de la suma de Gauss y la suma con bucle. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.18) por Luciano Moliterno (2025).

Pero este análisis generalmente es acertado para n pequeños, cuando trabajamos en el análisis de eficiencia teórica se trabaja con n tendiendo a infinito para resultados más certeros. Es por esto que se presenta el concepto de notación Big O para representar la complejidad de un algoritmo como una función que depende del tamaño de la entrada n .

Notación Big O

La base de la notación Big-O proporciona una forma sistemática y estandarizada de expresar la eficiencia temporal de un algoritmo en función del tamaño de entrada. Es un lenguaje matemático que simplifica la complejidad de los algoritmos, permitiendo evaluar y comparar de forma clara y concisa el rendimiento de diferentes algoritmos. La notación Big-O se centra en el peor de los casos y expresa el tiempo de ejecución asintótico a medida que la entrada aumenta hasta el infinito.

En la notación Big O se establece un límite superior en el crecimiento de una función de complejidad $T(n)$ en comparación con otra función de referencia $g(n)$. Esto nos ayuda a entender cómo se comporta $T(n)$ cuando el tamaño de la entrada (n) se vuelve muy grande.

En términos más formales, decimos que una función $T(n)$ es $O(g(n))$ si podemos encontrar dos constantes positivas (c) y (n_0) de modo que, para cualquier valor de (n) mayor o igual a (n_0), se cumpla la siguiente relación:

$$T(n) \leq c * g(n)$$

Esto implica que, para valores grandes de (n), $T(n)$ no crecerá más rápido que ($c * g(n)$), donde (c) es una constante que ajusta el crecimiento de $g(n)$. En otras palabras, Big O nos asegura que la función $T(n)$ no va a sobrepasar a $g(n)$ en el peor de los casos, proporcionando así una forma de medir su límite superior asintótico.

Por ejemplo, si tenemos dos algoritmos A y B con las siguientes funciones de tiempo:

$$T_a(n) = 3n^3$$

$$T_b(n) = 3n^3 + 2n^2 + 3n + 5$$

La función $T_b(n)$ es de orden superior n^3 por existen $n_0=10$, (c) = 4 , tales que $T_b(n) \leq c * n^3$, para todo $n \geq n_0$

Para buscar un límite superior a la función $T(n)$ hay que seguir los siguientes pasos:

- 1) Buscar el término que crece más rápido:

$$T_a(n) = 3n^3 \rightarrow 3n^3$$

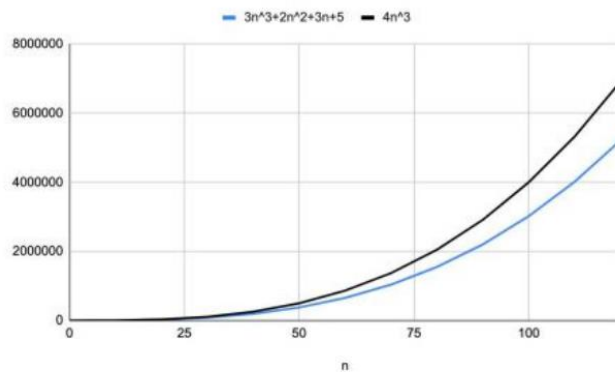
$$T_b(n) = 3n^3 + 2n^2 + 3n + 5 \rightarrow 3n^3$$

- 2) Eliminar su coeficiente:

$$3n^3 \rightarrow n^3$$

Figura 26

Gráfico de la función $T_b(n)$ y su cota superior



n	$3n^3+2n^2+3n+5$	$4n^3$
0	5	0
5	445	500
10	3235	4000
20	24865	32000
30	82895	108000
40	195325	256000
50	380155	500000
60	655385	864000
70	1039015	1372000
80	1549045	2048000
90	2203475	2916000
100	3020305	4000000
110	4017535	5324000
120	5213165	6912000

Nota. El gráfico compara el crecimiento de la función $T_b(n)$ y su cota superior para ilustrar que $T_b(n)$ es de orden $O(n^3)$. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.18) por Luciano Moliterno (2025).

Existen distintas notaciones Big O según el tipo de crecimiento de las funciones:

Figura 26

Tabla de funciones

notación	nombre
$O(1)$	constante
$O(\log n)$	logarítmico
$O(n)$	lineal
$O(n \log n)$	lineal-logarítmico
$O(n^2)$	cuadrática
$O(n^c)$	polinómico
$O(c^n)$	exponencial
$O(n!)$	factorial

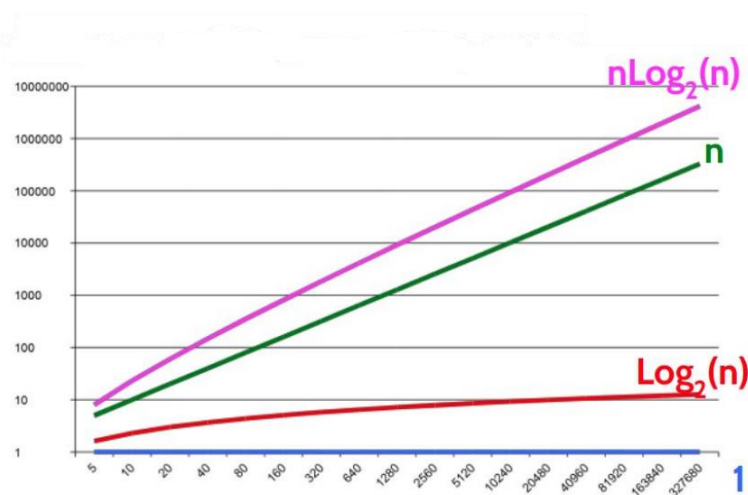
Nota. La imagen representa una tabla que clasifica las distintas notaciones Big O según el tipo de crecimiento de las funciones y su nombre correspondiente. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.23) por Luciano Moliterno (2025).

Cada notación Big O representa un orden de complejidad, entender estos órdenes nos permite predecir cómo escalarán los algoritmos y seleccionar la mejor solución para cada problema. Hay distintos órdenes de complejidad:

Los órdenes eficientes de complejidad, que resuelven problemas rápidamente, incluso con grandes entradas.

Figura 27

Órdenes eficientes de complejidad

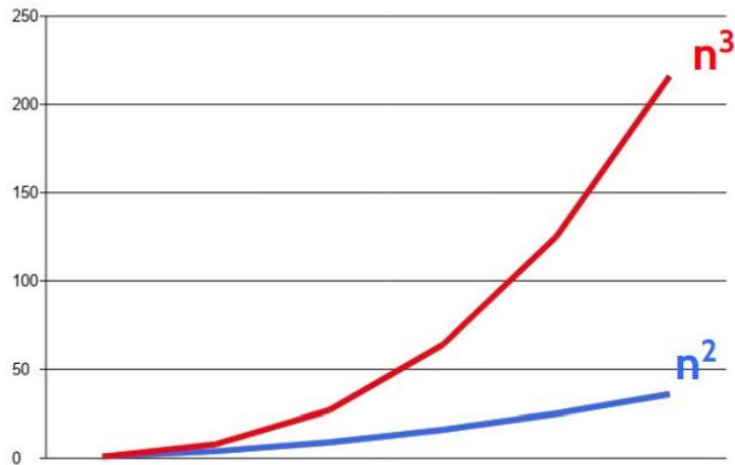


Nota. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.23) por Luciano Moliterno (2025)..

Los órdenes tratables de complejidad, si bien no son los mejores, son manejables para problemas de tamaño intermedio, pero menos eficientes para entradas muy grandes.

Figura 28

Órdenes intratables de complejidad

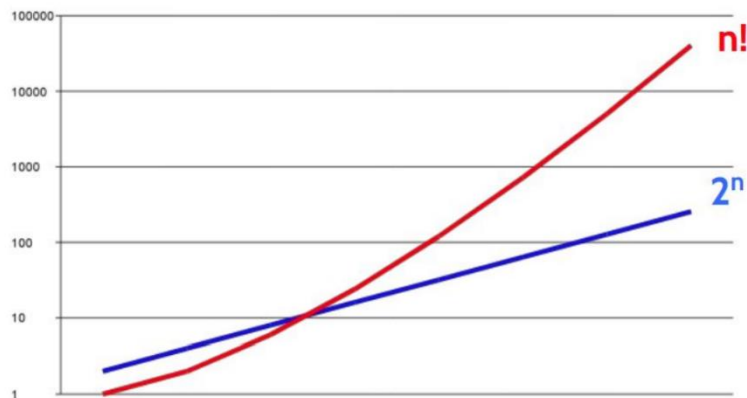


Nota. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.24) por Luciano Moliterno (2025).

Los órdenes intratables de complejidad, su crecimiento es tan rápido que se vuelven imposibles de ejecutar para grandes valores de n .

Figura 29

Órdenes intratables de complejidad



Nota. Tomado de *FTC-04 - Eficiencia de Algoritmos* (p.24) por Luciano Moliterno (2025).

Resolución Ejercicios

Construir algoritmos que tengan las complejidades de cada ejercicio. Luego encontrar que algoritmos tienen complejidad equivalente y cuáles no; y cual es el más complejo:

$$1) f(n) = 3n^2 + 5n + 2$$

$$2) f(n) = 2n^3 + 7n^2 + n$$

$$3) f(n) = 10n \log(n) + n^2$$

$$4) f(n) = 5n + \sqrt{n}$$

$$5) f(n) = 2^{n+1} + n^2$$

$$6) f(n) = 50 + n^3$$

$$1) f(n) = 3n^2 + 5n + 2$$

Figura 30

Algoritmo correspondiente a la función 1

```

1  def fun (n){
2      resul = 0 -----> = --> 1          #Inicializacion
3      for i in range(n){                  #Bucle Externo
4          for j in range(n){              #Inicializacion
5              resul += i + j -----> +;+= -> 3 #Suma
6          }
7      }
8      for i in range(n){                  #Bucle Externo
9          resul += i + 2 ----> +;+= -> 3      #Suma
10         resul *= 3 -----> *;= -> 2        #Multiplicacion
11     }
12     return resul -----> return --> 1     #Retorno
13 }
```

Nota. El algoritmo de la imagen toma un número n , realiza sumas y multiplicaciones sobre una variable `resul` en dos fases: primero con dos bucles anidados que suman valores de $i+j$, y luego en un segundo bucle donde se suman y multiplican valores, para finalmente devolver el resultado acumulado.

$$2) f(n) = 2n^3 + 7n^2 + n$$

Figura 31

Algoritmo correspondiente a la función 2

```

1  def fun (n){
2      for i in range(n){                #Bucle Externo
3          for j in range(n){            #Buble Interno
4              for k in range(n){        #Bucle Interno
5                  number = k -----> 1  #Inicializacion
6                  print(number) ----> 1  #Imprimir
7              }
8          }
9      }
10
11     for i in range(n){                #Bucle Externo
12         for j in range(n){            #Buble Interno
13             ope = 0 -----> 1          #Inicializacion
14             ope += j * I -----> 3      #Suma
15             ope -= I -----> 2          #Resta
16             print(ope) ----->1        #Imprimir
17         }
18     }
19
20     for i in range(n){                #Bucle
21         print(i)                      #Imprimir
22     }
23 }

```

Nota. En el algoritmo de la imagen se tienen tres bucles anidados imprimen el valor de k, seguido de dos bucles anidados realizan sumas y restas con i y j, e imprimen el resultado, y por último un bucle imprime el valor de i.

3) $f(n) = 10n \log(n) + n^2$

Figura 32

Algoritmo correspondiente a la función 3

```

1  Def fun (n){
2      For i in range(n){                #Bucle Externo
3          j = 1 -----> 1                #Inicializacion
4          resul = 1 -----> 1            #Inicializacion
5          while j < n { -----> 1        #Bucle Interno
6              resul += 1 * resul -----> 3 #Multiplicacion
7              print(resul - 2) -----> 2   #Imprimir
8              j *= 2 -----> 2            #Multiplicacion
9          }
10     }
11     For I in range(n){                #Bucle Externo
12         For j in range(n){            #Bucle Interno
13             Print(j) -----> 1          #Imprimir
14         }
15     }
16 }

```

Nota. El algoritmo de la imagen realiza operaciones de multiplicación y suma dentro de un bucle while, el cual está anidado dentro de un bucle for externo que controla las iteraciones, imprimiendo

los resultados en cada ciclo; después, usa dos bucles for anidados para imprimir los valores del índice j . La variable n determina el número de iteraciones tanto en el for como en el while. El crecimiento $O(\log(n))$ está dado por el *while* $j < n$, dentro de este j va aumentando su valor de forma logarítmica.

4) $f(n) = 5n + \sqrt{n}$

Figura 33

Algoritmo correspondiente a la función 4

```

1  Def fun(n){
2      For i in range(n){          #Bucle Externo
3          Resul + =1 ----> 2      #Suma
4          Resul -= 2 ----> 2      #Resta
5          Print(resul) ----> 1    #Imprimir
6      }
7
8      I = 0 -----> 1 pero sin valor          #Inicializacion
9      While i * I < n { -----> esto se hace por la base de la raiz, que seria 2 #Bucle Externo
10         I+=1----->2 pero no tiene valor    #Suma
11     }
12 }
13

```

Nota. En el algoritmo de la imagen se tiene un bucle que se repite en base a una condición cuadrática, imaginemos que $n = 100$, en un bucle normal tendríamos que hacer 100 iteraciones, con un iterador del tipo raíz solo tendríamos que hacer $10 = \sqrt{100}$, la cantidad de disminuiría significativamente.

5) $f(n) = 2^{n+1} + n^2$

Figura 34

Algoritmo correspondiente a la función 5

```

1  Def fun(n){
2      If n < 10 {           #Bucle Externo
3          |   Return n      #Return
4          |   }
5      For i in range(n){    #Bucle Externo
6          |   For j in range(n){ #Bucle Interno
7              |   Print(j)      #Imprimir
8          |   }
9      }
10
11     Return fun(n+1)        #Return
Q12 }

```

Nota. El algoritmo de la imagen define una función $fun(n)$ que verifica si n es menor que 10; si es así, devuelve n . Si n es 10 o mayor, ejecuta un bucle anidado donde imprime los valores de j . Al finalizar, la función se llama a sí misma con $n+1$, lo que genera una recursión hasta que n sea menor que 10.

6) $f(n) = 50 + n^3$

Figura 35

Algoritmo correspondiente a la función 6

```

1  Def fun (n){
2      Array[25] = {1,2,3,4,5,6,7,8,...,25}----->1 #Inicializacion
3      Array[1] +=1----->2                               #Suma
4      Array[2] +=2----->2
5      Array[3] +=3----->2
6      Array[4] +=4----->2
7      Array[5] +=5----->2
8      .
9      .
10     .
11     Array[25] =50----->1
12
13     For I in range(n){           #Bucle Externo
14         |   For j in range(n){    #Bucle Interno
15             |   For k in range(n){ #Bucle Interni
16                 |   Print(k)      #Imprimir
17             |   }
18         |   }
19     }
20 }

```

Nota. El algoritmo de la imagen inicializa un arreglo de 25 elementos, donde cada elemento i se incrementa en i . Luego, ejecuta tres bucles anidados que imprimen el valor de k .

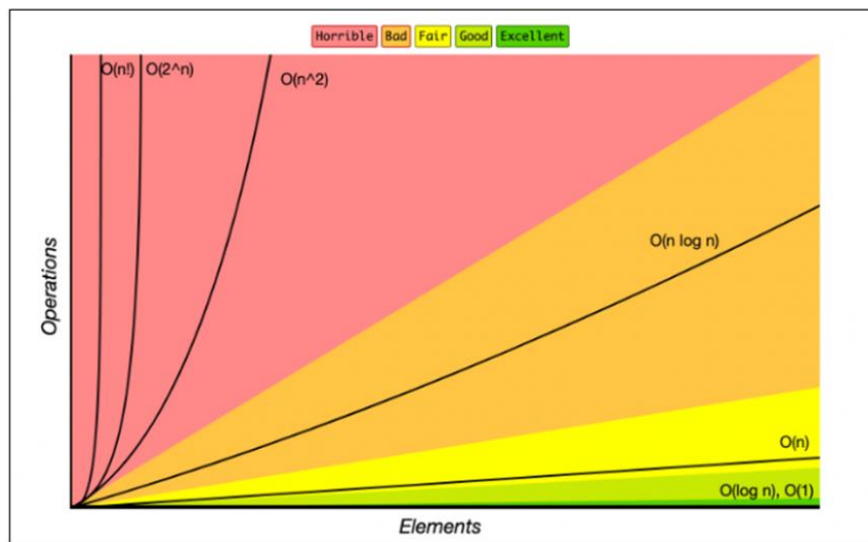
Se puede observar complejidad equivalente entre:

- Las funciones 1 y 3, dado que el término dominante de ambas es n^2 , $O(n^2)$.
- Las funciones 2 y 6, dado que el término dominante de ambas es n^3 , $O(n^3)$.

Por último, entre todas las complejidades que aparecieron en las diferentes funciones, la que mayor complejidad presenta es la función 5, con una complejidad de $O(2^2)$.

Figura 36

Gráfico de complejidad de tiempo



Nota. Tomado de *Freecodecamp*, por Joel Olawanle(5 de octubre de 2022).

<https://www.freecodecamp.org/news/big-o-cheat-sheet-time-complexity-chart/>

Referencias

- Kelley, Dean (1995). *Teoría de autómatas y lenguajes formales*. Prentice Hall. ISBN 0-13-518705-2.
- John Hopcroft, Rajeev Motwani, Jeffrey Ullman (2006). *Teoría de Autómatas, lenguajes y computación*.
- Jurado Málaga, Elena (2008). *Teoría de autómatas y lenguajes formales*. Universidad de Extremadura, Servicio de Publicaciones, ISBN 978-84-691-6345-0.
- Juan Manuel Cueva Lovelle, (2003) . *Lenguajes Gramáticas, y Autómatas en procesadores de lenguaje*. Servitec, ISBN: 84-688-4211.
- "Gramáticas." *Universidad Tecnológica Nacional, Facultad Regional Rosario*, 2015, https://www.frro.utn.edu.ar/repositorio/catedras/sistemas/2_anio/sintaxis/SySL_cap5_2015_Gram%C3%A1ticas.pdf.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2002). *Introducción a la teoría de autómatas, lenguajes y computación* (2a ed.). Madrid: Pearson Educación.
- Perez, Nicolás (2024). *FTC-04 - Eficiencia de Algoritmos*. https://campus.unla.edu.ar/aulas/pluginfile.php/1113247/mod_resource/content/1/FTC-04%20-%20Eficiencia%20de%20Algoritmos.pptx.pdf
- Parra, Ariel (2024). Big O (Complejidad de Algoritmos). Recuperado de https://cpc-gallos.github.io/Presentations/2024/2_001-BigO.pdf.
- Programación y Desarrollo de SW - UNLa. (14 de octubre de 2024). *FTC - Eficiencia y análisis de Algoritmos* [Archivo de Video] . Youtube. <https://www.youtube.com/watch?v=hNXwf1y9K30>

The University of Science & Technology. (4 de septiembre de 2024). *Notation Big-O*.

[https://msmk.university/big-o-](https://msmk.university/big-o-notation/#:~:text=%C2%BFQu%C3%A9%20es%20el%20Big%2DO,relativo%20del%20tama%C3%B1o%20de%20entrada)

[notation/#:~:text=%C2%BFQu%C3%A9%20es%20el%20Big%2DO,relativo%20del%20tama%C3%B1o%20de%20entrada](https://msmk.university/big-o-notation/#:~:text=%C2%BFQu%C3%A9%20es%20el%20Big%2DO,relativo%20del%20tama%C3%B1o%20de%20entrada).

[https://colab.research.google.com/drive/17fMMfYR4AICWOaW5kpTh2WB2XLmRrKvj?usp=](https://colab.research.google.com/drive/17fMMfYR4AICWOaW5kpTh2WB2XLmRrKvj?usp=sharing)
sharing